Cómo hacer el proyecto MVC de Tareas desde cero

Guía de estudio basada en **proyecto-mvc-tareas Alumnos** (PHP + PDO + MySQL + sesiones). Recopila las explicaciones de la conversación.

Parte 1 — El plan resumido

La idea es construirlo **de abajo hacia arriba**: base de datos, después modelo, luego controlador y al final vista. **Prueba cada paso antes de pasar al siguiente.**

Fase 0 — Entorno

1. Crea la estructura de carpetas vacía:

```
index.php
config/
app/models/   app/controllers/   app/views/{public,auth,user}/
public/css/
```

2. Arranca MySQL y comprueba que PHP funciona con un `index.php` que haga `echo "hola";` (`php -S localhost:8000`).

Fase 1 — Base de datos

3. `database.sql` con dos tablas: **usuarios** (id, nombre, email UNIQUE, password VARCHAR(255), fecha_registro) y **tareas** (id, titulo, descripcion, fase ENUM, estado ENUM, usuario_id **FK → usuarios** con ON DELETE CASCADE, fechas).
4. Ejecútalo e inserta un par de filas a mano en phpMyAdmin.

Fase 2 — Conexión

5. `config/database.php` con la clase `Database` y su método `getConnection()`, que devuelve un PDO con `ERRMODE_EXCEPTION` y utf8.
6. Pruébalo con un `SELECT * FROM usuarios` y un `var_dump`. **No sigas hasta que funcione.**

Fase 3 — Front controller (enrutador)

7. `index.php`: `session_start()`, los `require_once` y un `switch($_GET['action'] ?? 'home')`. Al principio cada `case` solo hace un `echo`.

Fase 4 — Parte pública

8. `PublicController::index()` → `Tarea::obtenerTodas()` (con JOIN para sacar el nombre del usuario) → vista `public/home.php`. Es tu **primer ciclo MVC completo**, y el resto repite este patrón.

Fase 5 — Autenticación

9. Modelo `Usuario`: `registrar()` (con `password_hash`), `emailExiste()`, `verificarPassword()` (con `password_verify`) y `obtenerPorId()`.

10. `AuthController`: `mostrarRegister` + `procesarRegister` (validaciones), `mostrarLogin` + `procesarLogin` (guardar en `$_SESSION`), y `logout`.
11. Mensajes flash en `$_SESSION['error']` / `$_SESSION['mensaje']`: se muestran en la vista y se hace `unset` justo después.
12. Prueba: registrarte, entrar, salir y equivocarte a propósito en cada validación.

Fase 6 — CRUD de tareas (rutas privadas)

13. **Listar**: `misTareas` → `Tarea::obtenerPorUsuario()` → `mis_tareas.php`.
14. **Crear**: `crearTarea` (formulario) + `guardarTarea` (POST → `Tarea::crear()` → redirigir).
15. **Editar**: `editarTarea&id=X` (cargar y **comprobar que es tuya**) + `actualizarTarea`.
16. **Eliminar**: `eliminarTarea&id=X` con `WHERE id=? AND usuario_id=?`.
17. Protege todas estas rutas: si no hay `$_SESSION['usuario_id']`, redirige al login. Mejora: escríbelo una sola vez en una función `requiereLogin()`.

Fase 7 — Seguridad

- Consultas preparadas en **todas** las consultas.
- `htmlspecialchars()` en **todo** lo que imprimas.
- Comprobar el dueño en editar y eliminar (prueba a cambiar el `id` en la URL con otro usuario).
- `.htaccess`: bloquear `config/` y `Options -Indexes`.

Fase 8 — Acabado

- `style.css` y una cabecera común (por ejemplo, `views/layout/header.php`).
- `README.md` con la instalación y los usuarios de prueba.
- Opcional: los ejercicios de `GUIA_MVC.md` (prioridad, filtro por estado, búsqueda, estadísticas).

> **Consejos:** una funcionalidad completa cada vez (modelo, controlador, vista y prueba). Haz commit al terminar cada fase. Para depurar, `var_dump($_POST); exit;` en el controlador.

Parte 2 — La guía explicada (para hacerlo sin chuletas)

La idea es que **entiendas por qué cada pieza está donde está**. Si lo entiendes, no tienes que memorizar el proyecto: lo vas sacando tú solo, porque cada paso te pide el siguiente.

1. La idea que tienes que llevar en la cabeza: el restaurante

Restaurante	Tu proyecto	Qué hace
Recepción de la puerta	`index.php`	Todo el mundo entra por aquí. Mira qué quieres (`?action=...`) y te manda a la mesa correcta.
Camarero	**Controlador**	Recibe tu pedido, comprueba que tiene sentido, pide a cocina lo que haga falta y te trae el plato. **No cocina ni emplata.**
Cocina / despensa	**Modelo**	El único que toca la base de datos. No sabe nada de HTML ni de formularios.
El plato servido	**Vista**	Solo HTML con algún `echo`. No decide nada, solo enseña lo que le dan.

Toda petición sigue siempre el mismo camino:

```
Navegador → index.php → Controlador → Modelo → BD
                                  ↓
                              Vista → Navegador
```

> Si te bloqueas, pregúntate: *"¿esto es decidir (controlador), guardar o leer datos (modelo) o enseñar (vista)?"*. Eso te dice en qué archivo va.

2. ¿Por qué se empieza por la base de datos?

Porque **cada capa depende de la que tiene debajo**: la vista necesita datos que le da el controlador, el controlador los pide al modelo, y el modelo necesita la BD. Si empiezas por las vistas, tienes formularios que no hacen nada y no puedes probarlos. De abajo arriba, **cada paso se puede probar en cuanto lo terminas**.

Paso 1 — Pensar las tablas (antes de escribir código)

¿Qué cosas existen en la aplicación? Usuarios y tareas, así que dos tablas. ¿Cómo se relacionan? Un usuario tiene muchas tareas y cada tarea es de un usuario. Por eso **la tarea guarda el id de su dueño** (`usuario_id`): la clave foránea.

> Truco: la FK siempre va en el lado del **"muchos"**. Una tarea tiene un solo dueño, así que la columna va en `tareas` .

- `email UNIQUE` : no puede haber dos cuentas con el mismo email.
- `password VARCHAR(255)` : el hash es largo. Nunca se guarda la contraseña tal cual.
- `ON DELETE CASCADE` : si se borra un usuario, se borran sus tareas (no quedan huérfanas).

- `ENUM` en fase y estado: solo se permiten unos valores fijos.

```
CREATE TABLE usuarios (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL
);

CREATE TABLE tareas (
  id INT AUTO_INCREMENT PRIMARY KEY,
  titulo VARCHAR(200) NOT NULL,
  descripcion TEXT,
  fase ENUM('Análisis','Diseño','Codificación','Pruebas','Despliegue') NOT NULL,
  estado ENUM('Pendiente','En Progreso','Completada') DEFAULT 'Pendiente',
  usuario_id INT NOT NULL,
  FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE CASCADE
);
```

> **Cómo lo compruebas:** inserta una tarea a mano. Si no te deja poner un `usuario_id` que no existe, la FK funciona.

Paso 2 — La conexión (config/database.php)

Por qué es un archivo aparte: todos los modelos necesitan conectarse. Si cada uno tuviera su propia conexión, cambiar la contraseña de MySQL te obligaría a tocar mil sitios.

```
class Database {
    public function getConnection() {
        $conn = new PDO("mysql:host=localhost;dbname=tareas_db;charset=utf8", "root", "");
        $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        return $conn;
    }
}
```

1. `new PDO("mysql:host=...;dbname=...", usuario, contraseña)` → conecta.
2. `ERRMODE_EXCEPTION` → si una consulta falla, **te lo grita** en lugar de fallar sin avisar.
3. `return` → se la das a quien la pida.

> **Cómo lo compruebas:** `$db = (new Database())->getConnection(); var_dump($db);`. Si sale `object(PDO)`, ya está.

Paso 3 — La recepción (index.php)

Por qué existe: con archivos sueltos (`login.php`, `tareas.php`...) repetirías en cada uno `session_start()`, los `require` y la comprobación de login. Con una única puerta de entrada lo haces **una vez**.

```
session_start();                          // 1. Despertar la sesión (siempre lo primero)
require_once 'config/database.php';       // 2. Cargar las piezas
require_once 'app/models/Tarea.php';      //    (todos los modelos y controladores)
// ...

$action = $_GET['action'] ?? 'home';      // 3. ¿Qué quiere el usuario?

switch ($action) {
    case 'home':  (new PublicController())->index(); break;
    case 'login': (new AuthController())->mostrarLogin(); break;
    // ...
    default: echo "Página no encontrada";
}
```

Por qué `session_start()` va lo primero: la sesión viaja en una cookie, y las cookies se envían en las cabeceras HTTP, **antes** del HTML. Si haces un `echo` antes, es demasiado tarde y PHP te da el aviso *"headers already sent"*.

Cómo lo compruebas: pon un `echo` en cada `case` y prueba `?action=login`, `?action=home` ... Cuando el enrutado va bien, cambias los `echo` por las llamadas a los controladores.

Paso 4 — Tu primer ciclo MVC completo: la portada

Es lo más importante del proyecto. **Si sabes hacer este, sabes hacer todos.**

Modelo: solo pregunta a la BD y devuelve datos.

```
class Tarea {
    private $conn;
    public function __construct($db) { $this->conn = $db; }

    public function obtenerTodas() {
        $sql = "SELECT t.*, u.nombre AS usuario_nombre
                FROM tareas t JOIN usuarios u ON t.usuario_id = u.id";
        $stmt = $this->conn->prepare($sql);
        $stmt->execute();
        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    }
}
```

Por qué el JOIN: la tabla `tareas` solo tiene el número del dueño (`usuario_id = 1`), pero en pantalla quieres poner "Juan". El JOIN va a buscar el nombre a la otra tabla.

Controlador: une las piezas.

```
class PublicController {
    public function index() {
        $db = (new Database())->getConnection();   // pide la conexión
        $tarea = new Tarea($db);                   // crea el modelo
        $tareas = $tarea->obtenerTodas();          // le pide datos
        require 'app/views/public/home.php';       // carga la vista
    }
}
```

> **Clave:** la vista se carga con `require` **dentro** del método, así que puede usar `$tareas` directamente. Así es como "le pasas" los datos a la vista.

Vista: recorre y pinta.

```
<?php foreach ($tareas as $t): ?>
  <div><?= htmlspecialchars($t['titulo']) ?> — <?= htmlspecialchars($t['usuario_nombre']) ?></div>
<?php endforeach; ?>
```

Paso 5 — Registro y login

La **sesión** es **una memoria que el servidor guarda para cada visitante** entre una página y otra. HTTP no recuerda nada por sí mismo. Al hacer login apuntas *"este es el usuario 3"*, y en las páginas siguientes lo miras.

Registro, el flujo mental:

1. Al llegar el POST, **valida antes de tocar la BD**: ¿campos vacíos?, ¿las contraseñas coinciden?, ¿al menos 6 caracteres?
2. ¿El email ya existe? → un `SELECT` en el modelo.
3. Si todo está bien → guardas **el hash**, nunca la contraseña.
4. Rediriges al login con un mensaje.

```
$hash = password_hash($password, PASSWORD_DEFAULT);   // al registrar
```

Login, el flujo mental: buscas al usuario **por email** → ¿existe? → ¿la contraseña cuadra con el hash? → lo apuntas en la sesión.

```
if (password_verify($passwordEscrita, $hashDeLaBD)) {
    $_SESSION['usuario_id'] = $usuario['id'];
    $_SESSION['usuario_nombre'] = $usuario['nombre'];
    header("Location: index.php?action=misTareas");
    exit();
}
```

> **Por qué `password_verify` y no `==`:** el hash nunca sale igual dos veces (lleva una parte aleatoria, la "sal"). `password_verify` sabe sacar la sal del hash y hacer la comprobación.

> **Por qué `exit()` después de `header("Location")`:** el `header` solo le dice al navegador "vete allí", pero **PHP sigue ejecutando** el resto del código. El `exit` lo para.

Logout:

```
session_destroy();
header("Location: index.php");
exit();
```

Mensajes "flash": después de una redirección las variables se pierden, porque es una petición nueva. Por eso se **guardan en la sesión, se enseñan una vez y se borran**.

```
$_SESSION['error'] = "Contraseña incorrecta";    // en el controlador
// en la vista:
if (isset($_SESSION['error'])) { echo $_SESSION['error']; unset($_SESSION['error']); }
```

Paso 6 — El CRUD de tareas

Orden: **Listar** → **Crear** → **Editar** → **Borrar**. Cada uno se apoya en el anterior.

Regla de oro: toda ruta privada empieza preguntando si hay alguien logueado.

```
if (!isset($_SESSION['usuario_id'])) {
    header("Location: index.php?action=login");
    exit();
}
```

Patrón "mostrar" y "procesar": crear y editar tienen dos acciones cada uno. `crearTarea` **muestra** el formulario (GET) y `guardarTarea` **lo recibe** (POST), guarda y redirige.

> **Por qué redirigir después de guardar:** si no lo haces y el usuario pulsa F5, se reenvía el POST y la tarea **se crea dos veces**. Esto se llama patrón PRG (Post → Redirect → Get).

Las 4 consultas que tienes que saber escribir:

```
SELECT * FROM tareas WHERE usuario_id = ?                                    -- listar las mías
INSERT INTO tareas (titulo, descripcion, fase, estado, usuario_id) VALUES (?,?,?,?,?)
UPDATE tareas SET titulo=?, descripcion=?, fase=?, estado=? WHERE id=? AND usuario_id=?
DELETE FROM tareas WHERE id=? AND usuario_id=?
```

> **El `AND usuario_id = ?` en UPDATE y DELETE es lo más importante de la seguridad del proyecto.** Sin eso, cualquiera logueado puede cambiar el `id` en la URL y borrar tareas de otra persona. El `usuario_id` sale **siempre de la sesión**, nunca del formulario.

Editar, explicado:

1. Te llega `?id=7` por GET.
2. Cargas esa tarea de la BD.
3. **Compruebas que es tuya.**
4. Muestras el formulario **ya relleno** (`value="<?= htmlspecialchars($tarea['titulo']) ?>"`) con un `<input type="hidden" name="id">`.
5. Al enviar → `actualizarTarea` hace el UPDATE y redirige.

Las 3 reglas de seguridad (y el porqué)

Regla	Qué evita	Cómo recordarlo
`prepare` + `?` en todas las consultas	**Inyección SQL**: alguien escribe `' OR 1=1 --` y entra sin contraseña	"Los datos del usuario **nunca** se pegan dentro del SQL"
`htmlspecialchars()` en todo lo que imprimas	**XSS**: alguien pone `<script>` como título y se ejecuta en el navegador de otros	"Todo lo que **sale** a pantalla se escapa"
`password_hash` / `password_verify`	Si roban la BD, no pueden leer las contraseñas	"Las contraseñas **nunca** se guardan como texto"

Resumen: lo que entra a la BD va con `prepare`, lo que sale a pantalla va con `htmlspecialchars`, y las contraseñas van con hash.

Lo mínimo que tienes que saber escribir sin mirar

1. `new PDO(...)` + `ERRMODE_EXCEPTION`
2. `$stmt = $conn->prepare($sql); $stmt->execute([...]); $stmt->fetchAll(PDO::FETCH_ASSOC);` (o `fetch()` si es una sola fila)
3. `session_start()`, guardar en `$_SESSION`, comprobar con `isset` y `session_destroy()`
4. `header("Location: ..."); exit();`
5. `password_hash` / `password_verify`
6. `foreach` + `htmlspecialchars` en la vista

Cómo practicar para hacerlo sin chuletas

1. **Primer intento:** siguiendo esta guía. Si te atascas, mira, pero **escribe tú, no copies y pegues**.
2. **Segundo intento (otro día):** desde una carpeta vacía y **sin mirar nada**. Apunta dónde te atascas: eso es lo que no tienes claro.
3. **Repasa solo esos puntos** y vuelve a empezar de cero.
4. **Tercer intento:** cronométrate y hazlo con otro tema (libros, coches...) para comprobar que entiendes el patrón y no te lo has aprendido de memoria.

> **Para el examen:** prueba cada paso en cuanto lo termines. Si lo escribes todo y pruebas al final, cuando falle no sabrás dónde está el error.

Parte 3 — Diferencias con el proyecto original

- **Los fragmentos de código están simplificados** para memorizarlos mejor. En el modelo del profe primero se rellenan las propiedades (`$tarea->usuario_id = ...`) y después se llama a `obtenerPorUsuario()`. Aquí los datos se pasan directamente. Las dos formas son correctas, pero en el examen usa la que os haya enseñado el profe.
- **La tabla SQL de esta guía es más corta**: no incluye `fecha_registro`, `fecha_creacion`, `fecha_actualizacion` ni los índices. La versión completa está en el `database.sql` del proyecto.